Project - High Level Design on LogiDoc AI

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrollment Number(s):

Sr no	Student Name	Enrollment Number
1.	Vasu Joshi	EN22EL301063
2.	Yash Kag	EN22CS3011103
3.	Yash Parashar	EN22CS3011106
4.	Yashee Verma	EN22CS3011115
5.	Yashraj Singh Sisodiya	EN22CS3011120

Group Name: Group 12D10

Project Number: DO-25

Industry Mentor Name:

University Mentor Name: Prof. Divya Kumawat

Academic Year: 2025-26

Table of Contents

S.No.	Title	Page No.
1	Introduction	
	1.1 Scope of the document	3
	1.2 Intended Audience	3
	1.3 System Overview	3
2	System Design	3
	2.1 Application Design	4
	2.2 Process Flow	4
	2.3 Information Flow	5
	2.4 Components Design	5
	2.5 Key Design Considerations	6
	2.6 API Catalogues	6
3	Data Design	6
	3.1 Data Model	6
	3.2 Data Access Mechanism	6
	3.3 Data Retention Policies	6
	3.4 Data Migration	7
4	Interfaces	7
5	State And Session Management	7
6	Caching	7
7	Non – Functional Requirements	7
	7.1 Security Aspects	7
	7.2 Performance Aspects	7
8	References	7

1. Introduction:

1.1 Scope of the Document:

This document provides a detailed description of:

- System architecture and design
- Data flow and processing mechanisms
- API structure and integration
- Security, performance, and scalability considerations

1.2 Intended Audience:

- Developers and software engineers
- Project evaluators and academic reviewers
- System architects
- AI/ML engineers

1.3 System Overview:

LogiDoc-AI is a **full-stack AI-powered application** consisting of:

- Frontend interface for user interaction
- Backend API for processing
- AI/ML pipeline for document understanding
- Vector database for semantic search

It processes logistics documents and provides intelligent query responses using a RAG pipeline.

2. System Design:

Architecture Layers:

1. Presentation Layer (Frontend)
2. Application Layer (Backend API)
3. AI Processing Layer (RAG + LLM)
4. Data Layer (Vector DB + Storage)

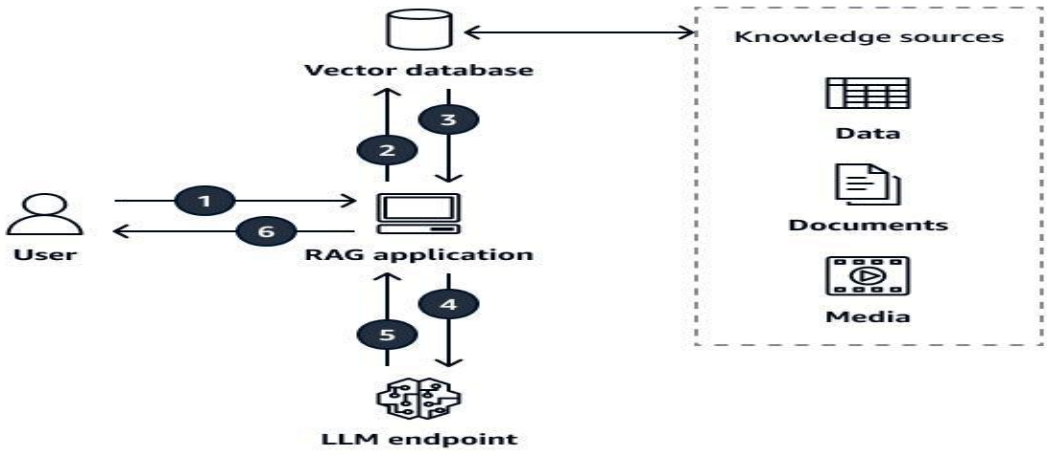

Fig 1: System Design

2.1 Application Design:

- Built using **component-based architecture**
- Backend follows **RESTful API design**
- Asynchronous processing using FastAPI
- Modular AI pipeline for scalability

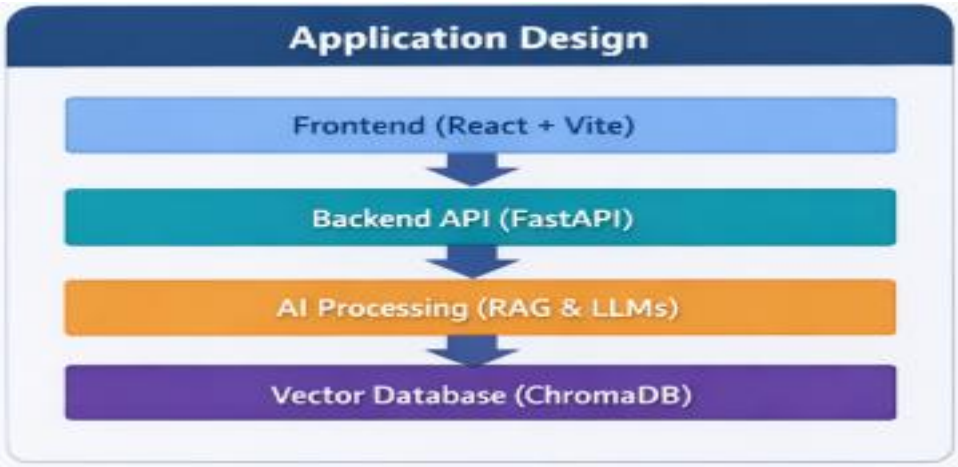

Fig 2: Application Design

2.2 Process Flow:

- User uploads document
- System validates logistics relevance
- Text is extracted and preprocessed
- Data is converted into embeddings
- Stored in vector database
- User query processed
- Relevant data retrieved
- LLM generates final response

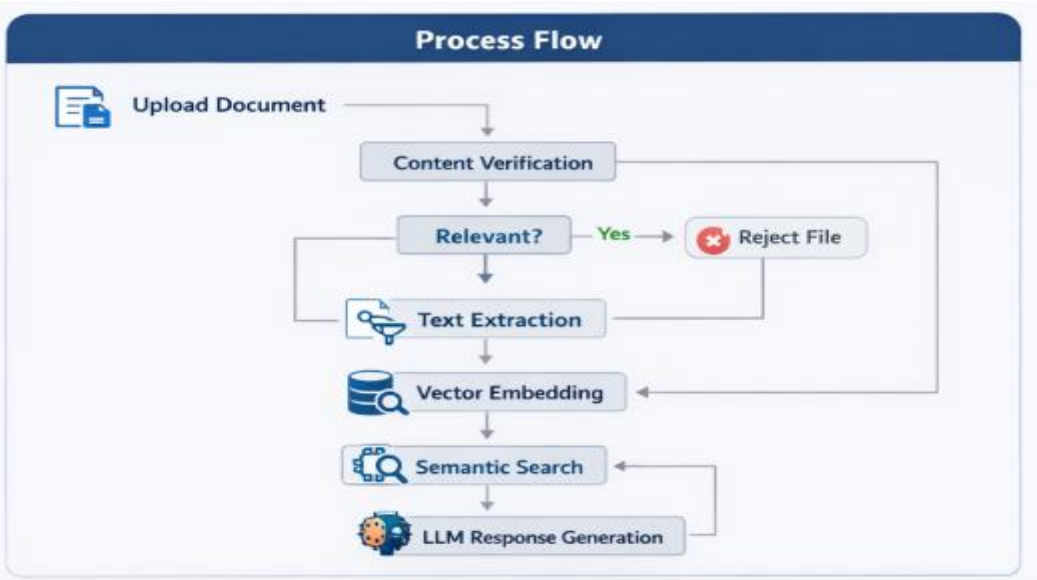

Fig 3: Process Flow

2.3 Information Flow:

- Input: Documents (PDF/Text)
- Processing: Validation → Chunking → Embedding
- Storage: Vector DB
- Output: AI-generated responses

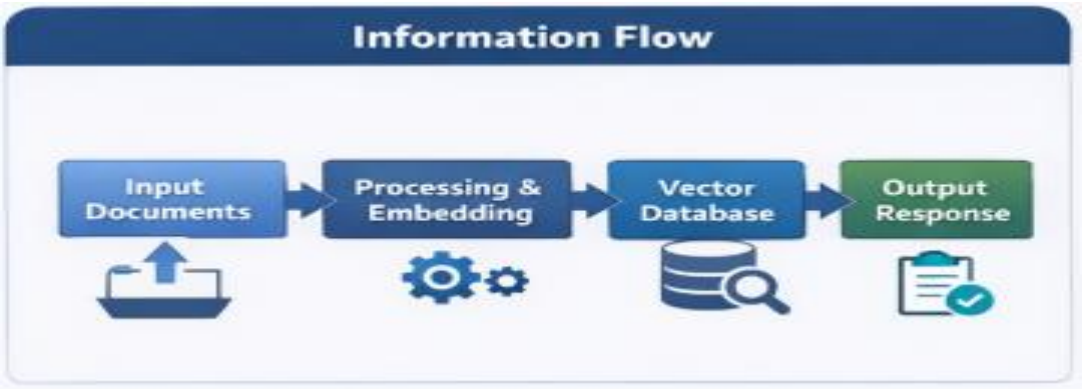

Fig 4: Information Flow.

2.4 Components Design:

Key Components:

- Frontend UI (React + Vite)
- Backend API (FastAPI)
- Validation Module (AI filter)
- Embedding Generator
- Vector Store (ChromaDB)
- LLM Integration Module

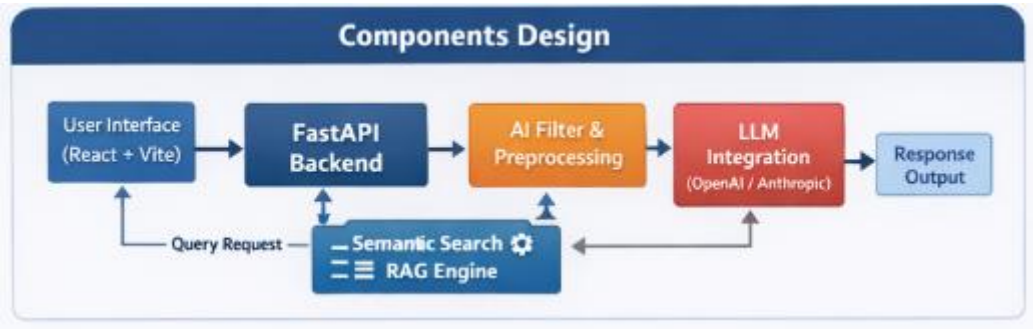

Fig 5: Components Design

2.5 Key Design Considerations:

- Scalability for large document volumes
- Accuracy in document classification
- Low latency in query response
- Modularity for future enhancements
- Fault tolerance and error handling

2.6 API Catalogue:

Example APIs:

- POST /upload → Upload document
- POST /validate → Check logistics relevance
- POST /query → Ask questions
- DELETE /workspace → Clear database

3. Data Design:

3.1 Data Model:

- Document Entity
- Chunk Entity
- Embedding Vector
- Query-Response Mapping

3.2 Data Access Mechanism:

- REST APIs for frontend interaction
- Direct vector search in database
- Async data retrieval

3.3 Data Retention Policies:

- Temporary storage of processed documents
- User-controlled deletion via workspace manager
- No long-term storage unless required

3.4 Data Migration:

- Supports re-indexing of documents
- Future support for cloud migration

4. Interfaces:

- User Interface (Web App)
- API Interface (Backend)
- External APIs (LLM providers

5. State and Session Management:

- Stateless backend (REST APIs)
- Session handled at frontend level
- Token-based interaction (if implemented)

6. Caching:

- Frequently accessed queries cached
- Improves response time
- Reduces LLM API calls.

7. Non-Functional Requirements:

- High availability
- Scalability
- Reliability
- Maintainability
- Usability

7.1 Security Aspects:

- Input validation
- Secure API communication (HTTPS)
- API key protection for LLM services
- Access control mechanisms

7.2 Performance Aspects:

- FastAPI ensures high throughput
- Vector search enables quick retrieval
- Async processing reduces latency
- Optimized chunking improves accuracy

8. References:

- OpenAI API documentation
- Anthropic API documentation
- ChromaDB documentation
- FastAPI official docs
- Research papers on RAG (Retrieval-Augmented Generation)